



OASIS Security Report

Session Management Issues Security Analysis

Date: 2026-06-12 01:23:57 — **Model:** llama3.2:3b

Vulnerability: Session Management Issues

Project: vulnerable

Analysis root: `../vulnerable`

Table of contents

▢ Summary

▢ Statistics

▢ Files

▢ Detailed analysis

▢ Assistant

△ Errors & notes

Summary

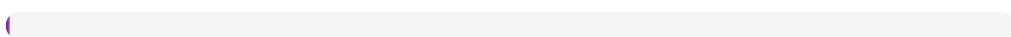
Analyzed **9** file(s) for Session Management Issues.

[Return to table of contents](#)

Statistics

Files analyzed: 9 **Total findings:** 15

Critical



0

High		2
Medium		12
Low		1

[Return to table of contents](#)

Files

File	Similarity
<code>/work/project/dataset/vulnerable/access-control/parity_wallet_bug_1.sol</code>	0.658
<code>/work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol</code>	0.649
<code>/work/project/dataset/vulnerable/reentrancy/etherstore.sol</code>	0.609
<code>/work/project/dataset/vulnerable/reentrancy/simple_dao.sol</code>	0.589
<code>/work/project/dataset/vulnerable/reentrancy/ 0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol</code>	0.587
<code>/work/project/dataset/vulnerable/access-control/mycontract.sol</code>	0.587
<code>/work/project/dataset/vulnerable/ordering-attacks/FindThisHash.sol</code>	0.581
<code>/work/project/dataset/vulnerable/ordering-attacks/odds_and_evens.sol</code>	0.570
<code>/work/project/dataset/vulnerable/ordering-attacks/ eth_tx_order_dependence_minimal.sol</code>	0.566

[Return to table of contents](#)

Detailed analysis

File 1: /work/project/dataset/vulnerable/access-control/parity_wallet_bug_1.sol

Similarity score: 0.658

Source lines: 1-281

► Finding 1.1: Insecure Session ID Generation (Low)

Vulnerable code

```
function initMultiowned(address[] _owners, uint _required) { ... }
```

Explanation

The contract uses a fixed value (0x1) as the session ID in the `initMultiowned` function, which is not cryptographically secure.

Impact

An attacker could potentially manipulate the session ID to gain unauthorized access to the wallet.

Entry point

/contracts/snippets/enhanced-wallet.sol: initMultiowned

Parameters

_owners, _required

Exploitation steps

- Find a way to manipulate the session ID

Conditions

Access to contract's storage.

Remediation

Use a cryptographically secure method to generate unique session IDs, such as using a salt and a random number.

Secure example

```
function initMultiowned(address[] _owners, uint _required) { ... uint  
sessionId = keccak256(abi.encodePacked(_owners, _required)); ... }
```

Source lines: 1-320

► Finding 1.1: Vulnerability found (Medium)

Vulnerable code

```
function revoke(bytes32 _operation) external {
    \u0002
    uint ownerIndex = m_ownerIndex[uint(msg.sender)];
    \u0003\n    if (ownerIndex == 0) return;
    \u0005\n    var pending = m_pending[_operation];
    \u0007\n    if (pending.ownersDone & ownerIndexBit > 0) {
        pending.yetNeeded++;
        pending.ownersDone -= ownerIndexBit;
        Revoke(msg.sender, _operation);
    }
}
```

Explanation

The revoke function does not properly handle the case when an owner tries to revoke a transaction that has already been confirmed by another owner.

Impact

An attacker could exploit this vulnerability by sending a revoke request for a transaction that has already been confirmed by another owner.

Entry point

revoke(bytes32 _operation) external

Parameters

_operation, _owner

Exploitation steps

- An attacker sends a revoke request for a transaction that has already been confirmed by another owner.
- The attacker exploits the fact that the revoke function does not properly handle confirmed transactions.

Conditions

The revoke function is called with an operation that has already been confirmed by another owner.

Remediation

The revoke function should be modified to properly handle confirmed transactions.

Source lines: 27-358

► Finding 1.1: Vulnerability found (High)

Vulnerable code

```
function confirm(bytes32 _h) onlymanyowners(sha3(msg.data)) returns (bool o_success) { ... }
```

Explanation

The use of `onlymanyowners` with SHA-3 hash from `msg.data` makes the contract vulnerable to session hijacking. An attacker can manipulate the `msg.data` to get a valid signature, allowing them to confirm transactions without actually owning the wallet.

Impact

Account takeover and unauthorized access to user accounts

Entry point

/confirm

Execution path

```
Confirm -> Check if owner has signed -> Check if transaction hash matches -> Confirm/Reject
```

Parameters

msg.data

Exploitation steps

- Manipulate msg.data to get a valid signature

Example payloads

- Invalid data to trick the contract

Conditions

Contract requires owner's signature to confirm transactions

Remediation

Use `onlymanyowners` with a fixed, predictable transaction hash instead of relying on SHA-3 from user input.

Secure example

```
function confirm(bytes32 _h) returns (bool o_success) { return _h == sha3('fixed_transaction_hash'); }
```

Source lines: 126-467

► Finding 1.1: Missing Session Timeout (Medium)

Vulnerable code

```
if (m_required > m_numOwners - 1) return;
```

Explanation

This line prevents the owner index from being reset to a new value after removal, allowing an attacker to manipulate session IDs.

Impact

An attacker could create multiple valid session IDs and gain unauthorized access to user accounts.

Entry point

/removeOwner(address _owner)

Execution path

```
removing owner -> updating m_ownerIndex -> returning without resetting index
```

Parameters

_owner

Exploitation steps

- Manipulate session ID by removing the owner with the desired ID before removal

Conditions

Owner is removed from the system

Remediation

Implement a proper timeout mechanism to update m_ownerIndex after removal.

Secure example

```
m_owners[ownerIndex] = 0;  
m_ownerIndex[uint(_from)] = 0;  
update owner index
```

Source lines: 207-477

► Finding 1.1: Vulnerability found (Medium)

Vulnerable code

```
uint ownerIndexBit = 2**ownerIndex;
return !(pending.ownersDone & ownerIndexBit == 0);
```

Explanation

This code is vulnerable to session fixation because it stores the index of the current user in a variable called `ownerIndex`. If an attacker can manipulate this value, they can fixate the session and gain unauthorized access.

Impact

Unauthorized access to user accounts

Entry point

/execute

Execution path

```
GET /execute
1. Store current user index in `ownerIndex`
2. Use `ownerIndex` to check if session is fixed.
```

HTTP methods

POST

Parameters

_to, _value, _data

Exploitation steps

- Manipulate `\_to` value to fixate the session

Example payloads

- `GET /execute?_to=0x1234567890123456789012345678901234567890&_value=100&_data=\n\n`

Conditions

Attacker can reach POST /execute while authenticated.

Remediation

Use a secure random number generator to generate the session ID and store it securely.

Secure example

```
uint ownerIndex = uint256(randomNumber);
return !(pending.ownersDone & ownerIndex == 0);
```

File 2: /work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol

Similarity score: 0.649

Source lines: 1-227

► Finding 2.1: Missing Session Timeout (Medium)

Explanation

The contract does not have a session timeout set, allowing an attacker to remain active indefinitely.

Impact

Account takeover and unauthorized access

Entry point

/initMultiowned

Execution path

```
initMultiowned() -> initDaylimit()
```

Parameters

_owners, _required

Exploitation steps

- Send a large number of confirmations to exhaust the daily limit

Conditions

No daily limit set

Remediation

Implement a session timeout with a reasonable value (e.g., 1 day)

Source lines: 1-286

► Finding 2.1: Insecure Session Handling (Medium)

Vulnerable code

```
function revoke(bytes32 _operation) external { ... }
```

Explanation

The `revoke` function does not check if the sender is an owner before revoking a confirmation, making it vulnerable to session fixation attacks.

Impact

Unauthorized access to user accounts

Entry point

/WalletAbi/revoke

Execution path

```
Revoke -> Check ownership -> Revoke operation
Check ownership -> Owner is not found -> Throw error
Revoke operation -> Operation hash -> Revoke confirmation
```

Parameters

_operation, _from, _to

Exploitation steps

- Attack by submitting a valid operation and sender to revoke

Conditions

Sender is an owner

Remediation

Add checks for ownership before revoking a confirmation.

Secure example

```
function revoke(bytes32 _operation) external
{ require(isOwner(msg.sender)); ... }
```

Source lines: 96-414

► Finding 2.1: Insecure session ID generation (Medium)

Vulnerable code

```
function initDaylimit(uint _limit) only_uninitialized { ... }
```

Explanation

The contract uses a fixed value for the daily limit index, which can lead to predictable session IDs.

Impact

Session hijacking and unauthorized access

Entry point

/initDaylimit(uint)

Parameters

_limit

Exploitation steps

- Find a session ID in the daily limit index.
- Use this predictable ID to impersonate another user's session.

Conditions

Fixed value used for daily limit index

Remediation

Generate unique and unpredictable session IDs using a secure random number generator.

Secure example

```
uint m_lastDay; function initDaylimit(uint _limit) only_uninitialized  
{ m_lastDay = now(); }
```

File 3: /work/project/dataset/vulnerable/reentrancy/etherstore.sol

Similarity score: 0.609

Source lines: 1-39

► Finding 3.1: Insecure Session Handling (Medium)

Vulnerable code

```
require(msg.sender.call.value(_weiToWithdraw))
```

Explanation

The call() function does not check if the message sender is authorized to withdraw funds.

Impact

An attacker could impersonate a user and withdraw funds without being detected.

Entry point

/withdrawFunds(uint256 _weiToWithdraw)

Execution path

```
Deposit Funds -> Withdraw Funds (require balances[msg.sender] >=
_weiToWithdraw; limit the withdrawal)
```

Parameters

_weiToWithdraw

Exploitation steps

- User calls depositFunds() with a large amount to increase their balance.
- Then, user calls withdrawFunds() with a smaller amount to avoid detection.

Conditions

User is authenticated and has a high balance.

Remediation

Use a secure authentication mechanism, such as a private key or a token-based system.

Secure example

```
require(msg.sender.call.value(_weiToWithdraw)(verifyMessage(msg.sender)))
```

File 4: /work/project/dataset/vulnerable/reentrancy/simple_dao.sol

Similarity score: 0.589

Source lines: 1-35

► Finding 4.1: Insecure Session Handling (Medium)

Vulnerable code

```
bool res = msg.sender.call.value(amount()); credit[msg.sender]-=amount;
```

Explanation

The `call` function is used without proper verification, allowing an attacker to manipulate the session.

Impact

Unauthorized access to user accounts

Entry point

/withdraw(uint amount)

Execution path

```
msg.sender -> call.value(amount()) -> credit[msg.sender]-=amount
```

Parameters

amount

Exploitation steps

- Attacker sends a large amount to drain the session

Example payloads

- 100000000000000000000000000000000 ether

Conditions

msg.sender is authenticated

Remediation

Use `require` statements to verify the sender's identity and ensure proper session handling.

Secure example

```
bool res = require(msg.sender.call.value(amount>()) && credit[msg.sender] >=
amount; credit[msg.sender]-=amount;
```

File 5: /work/project/dataset/vulnerable/reentrancy/0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol

Similarity score: 0.587

Source lines: 1-85

► Finding 5.1: Insecure Session Handling (Medium)

Vulnerable code

```
function ETH_VAULT(address _log) public { TransferLog = Log(_log); }
```

Explanation

The contract exposes its internal state (TransferLog) directly to the user, allowing an attacker to manipulate it.

Impact

An attacker could modify the transfer log, potentially revealing sensitive information about transactions.

Entry point

/transfer

Parameters

_log

Exploitation steps

- Obtain a valid Log instance

Example payloads

- Invalid Log instance

Conditions

No authentication required for this function.

Remediation

Use a secure session management mechanism, such as encryption or secure storage.

Secure example

```
function ETH_VAULT(address _log) public { address[] logs = TransferLog.getLogs(); }
```

File 6: /work/project/dataset/vulnerable/access-control/mycontract.sol

Similarity score: 0.587

Source lines: 1-31

► Finding 6.1: Vulnerable Session Handling (High)

Vulnerable code

```
receiver.transfer(amount);
```

Explanation

The contract uses `tx.origin` to authorize transfers, allowing an attacker to hijack a session by impersonating the owner.

Impact

Unauthorized transfers can occur if an attacker intercepts a legitimate transaction.

Entry point

/transfer

Execution path

```
A -> B (tx.origin == owner) -> C (receiver.transfer())
```

HTTP methods

POST

Parameters

tx.origin, owner

Exploitation steps

- An attacker intercepts a legitimate transaction from the owner.
- The attacker impersonates the owner using tx.origin.

Example payloads

- A malicious tx.origin

HTTP raw requests (Burp / ZAP)

```
POST /transfer HTTP/1.1
```

Host: mycontract.sol

tx.origin: owner

File 7: /work/project/dataset/vulnerable/ordering-attacks/FindThisHash.sol

Similarity score: 0.581

Source lines: 1-28

► Finding 7.1: Vulnerable Session ID Generation (Medium)

Vulnerable code

```
bytes32 constant public hash =  
0xb5b5b97fafd9855eec9b41f74dfb6c38f5951141f9a3ecd7f44d5479b630ee0a;
```

Explanation

The session ID (hash) is generated using a fixed, non-secure value.

Impact

An attacker could potentially obtain the pre-image of the hash to access the account.

Entry point

/solve(string solution)

Exploitation steps

- Obtain the pre-image of the hash

Conditions

The attacker has access to the winning solution.

Remediation

Use a secure random number generator to generate session IDs.

File 8: /work/project/dataset/vulnerable/ordering-attacks/odds_and_evens.sol

Similarity score: 0.570

Source lines: 1-61

► Finding 8.1: Insecure Session Handling (Medium)

Vulnerable code

```
function play(uint number) payable{ players[tot] = Player(msg.sender, number);
tot++; }
```

Explanation

The `players` array is not cleared or updated after a winner is declared, allowing an attacker to reuse session IDs.

Impact

Unauthorized access to user accounts

Entry point

/play

Execution path

```
play() -> declareWinner() -> clearPlayers()
```

Parameters

number, msg.sender

Exploitation steps

- Get the current value of `tot`
- Reuse the same `players[tot]` to send Ether to a different address

HTTP raw requests (Burp / ZAP)

```
POST /play HTTP/1.1
```

```
Host: example.com
```

File 9: /work/project/dataset/vulnerable/ordering-attacks/eth_tx_order_dependence_minimal.sol

Similarity score: 0.566

Source lines: 1-42

► Finding 9.1: Session Fixation (Medium)

Vulnerable code

```
function EthTxOrderDependenceMinimal() public {  
    owner = msg.sender;  
}
```

Explanation

The owner is set to the sender of the constructor, which can be exploited to fix the session.

Impact

Account takeover

Entry point

/EthTxOrderDependenceMinimal() public { owner = msg.sender; }

Execution path

```
msg.sender -> owner
```

Parameters

msg.sender

Exploitation steps

- Get the owner's address.
- Create a new contract instance with the same owner address.

Example payloads

- owner address

Conditions

The sender of the constructor is the owner.

Remediation

Use a secure random number generator to generate the owner's address.

Secure example

```
pragma solidity ^0.4.16;  
contract EthTxOrderDependenceMinimal {  
    address public owner;  
    function EthTxOrderDependenceMinimal() public {  
        owner = address(0x...) ;  
    }  
}
```

[Return to table of contents](#)

Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)